

JOIN Observation

Tables used: STUDENT, FACULTY, DEPT, COURSE, ENROLLMENT.

AIM: To combine rows from two or more tables using various types of JOIN operations in SQL.

-- 1. INNER JOIN: Only matching rows from both tables

```
-- Courses along with the name of the faculty teaching them
SELECT c.CourseNo, c.CourseDesc, f.FacName
FROM COURSE c
INNER JOIN FACULTY f ON c.FacNo = f.FacNo;
```

```
mysql> -- Courses along with the name of the faculty teaching them
mysql> SELECT c.CourseNo, c.CourseDesc, f.FacName
      -> FROM COURSE c
      -> INNER JOIN FACULTY f ON c.FacNo = f.FacNo;
+-----+-----+-----+
| CourseNo | CourseDesc | FacName |
+-----+-----+-----+
| C01      | DBMS       | Ramesh  |
| C02      | OS         | Mohan   |
| C03      | DataStructur | Arjun   |
| C04      | DBMS Lab   | Ramesh  |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
-- Students along with the courses they are enrolled in
SELECT s.Name, c.CourseDesc, e.EnrollDate
FROM STUDENT s
INNER JOIN ENROLLMENT e ON s.RegNo = e.RegNo
INNER JOIN COURSE c ON e.CourseNo = c.CourseNo;
```

```
mysql> -- Students along with the courses they are enrolled in
mysql> SELECT s.Name, c.CourseDesc, e.EnrollDate
      -> FROM STUDENT s
      -> INNER JOIN ENROLLMENT e ON s.RegNo = e.RegNo
      -> INNER JOIN COURSE c ON e.CourseNo = c.CourseNo;
+-----+-----+-----+
| Name      | CourseDesc | EnrollDate |
+-----+-----+-----+
| Aravind Kumar | DBMS       | 2026-06-01 |
| Divya      | DBMS       | 2026-06-02 |
| Aravind Kumar | OS         | 2026-06-01 |
| Karthik     | DataStructur | 2026-06-02 |
| Meena       | DBMS Lab   | 2026-06-03 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

-- 2. LEFT JOIN: All rows from the left table, matched or not

```
-- Every department, along with its head faculty's name (if any)
SELECT d.DeptName, f.FacName AS HeadName
FROM DEPT d
LEFT JOIN FACULTY f ON d.DeptHead = f.FacNo;
```

```
mysql> -- Every department, along with its head faculty's name (if any)
mysql> SELECT d.DeptName, f.FacName AS HeadName
      -> FROM DEPT d
      -> LEFT JOIN FACULTY f ON d.DeptHead = f.FacNo;
+-----+-----+
| DeptName | HeadName |
+-----+-----+
| CSE      | Ramesh   |
| ECE      | Sunitha  |
| IT       | Arjun    |
+-----+-----+
3 rows in set (0.00 sec)
```

```
-- Every student, and the courses they've enrolled in (NULL if none)
SELECT s.Name, c.CourseDesc
FROM STUDENT s
LEFT JOIN ENROLLMENT e ON s.RegNo = e.RegNo
LEFT JOIN COURSE c ON e.CourseNo = c.CourseNo;
```

```
mysql> -- Every student, and the courses they've enrolled in (NULL if none)
mysql> SELECT s.Name, c.CourseDesc
      -> FROM STUDENT s
      -> LEFT JOIN ENROLLMENT e ON s.RegNo = e.RegNo
      -> LEFT JOIN COURSE c ON e.CourseNo = c.CourseNo;
+-----+-----+
| Name          | CourseDesc |
+-----+-----+
| Aravind Kumar | DBMS       |
| Aravind Kumar | OS         |
| Divya         | DBMS       |
| Karthik       | DataStructur |
| Meena         | DBMS Lab   |
| Suresh        | NULL       |
+-----+-----+
6 rows in set (0.00 sec)
```

-- 3. RIGHT JOIN: All rows from the right table, matched or not

```
-- Every faculty member, and any course they teach (NULL if none)
SELECT f.FacName, c.CourseDesc
FROM COURSE c
RIGHT JOIN FACULTY f ON c.FacNo = f.FacNo;
```

```
mysql> -- Every faculty member, and any course they teach (NULL if none)
mysql> SELECT f.FacName, c.CourseDesc
      -> FROM COURSE c
      -> RIGHT JOIN FACULTY f ON c.FacNo = f.FacNo;
```

FacName	CourseDesc
Ramesh	DBMS
Ramesh	DBMS Lab
Mohan	OS
Sunitha	NULL
Kavya	NULL
Arjun	DataStructur

6 rows in set (0.00 sec)

-- 4. FULL OUTER JOIN (simulated with UNION in MySQL)

MySQL has no FULL OUTER JOIN keyword — combine LEFT and RIGHT JOIN with UNION to get the same result.

```
SELECT f.FacName, c.CourseDesc
FROM FACULTY f
LEFT JOIN COURSE c ON f.FacNo = c.FacNo
UNION
SELECT f.FacName, c.CourseDesc
FROM FACULTY f
RIGHT JOIN COURSE c ON f.FacNo = c.FacNo;
```

```
mysql> SELECT f.FacName, c.CourseDesc
      -> FROM FACULTY f
      -> LEFT JOIN COURSE c ON f.FacNo = c.FacNo
      -> UNION
      -> SELECT f.FacName, c.CourseDesc
      -> FROM FACULTY f
      -> RIGHT JOIN COURSE c ON f.FacNo = c.FacNo;
```

FacName	CourseDesc
Ramesh	DBMS Lab
Ramesh	DBMS
Mohan	OS
Sunitha	NULL
Kavya	NULL
Arjun	DataStructur

6 rows in set (0.03 sec)

-- 5. SELF JOIN: A table joined with itself

```
-- Pair up faculty members who belong to the same department
SELECT a.FacName AS Faculty1, b.FacName AS Faculty2, a.Dept
FROM FACULTY a
JOIN FACULTY b ON a.Dept = b.Dept AND a.FacNo < b.FacNo;
```

```
mysql> -- Pair up faculty members who belong to the same department
mysql> SELECT a.FacName AS Faculty1, b.FacName AS Faculty2, a.Dept
-> FROM FACULTY a
-> JOIN FACULTY b ON a.Dept = b.Dept AND a.FacNo < b.FacNo;
```

Faculty1	Faculty2	Dept
Ramesh	Mohan	CSE
Ramesh	Arjun	CSE
Mohan	Arjun	CSE

```
3 rows in set (0.00 sec)
```

-- 6. CROSS JOIN: Every row combined with every row (Cartesian product)

```
-- Every faculty member paired with every course (for scheduling checks etc.)
SELECT f.FacName, c.CourseDesc
FROM FACULTY f
CROSS JOIN COURSE c;
```

```
mysql> SELECT f.FacName, c.CourseDesc
-> FROM FACULTY f
-> CROSS JOIN COURSE c;
```

FacName	CourseDesc
Ramesh	DBMS Lab
Ramesh	DataStructur
Ramesh	OS
Ramesh	DBMS
Mohan	DBMS Lab
Mohan	DataStructur
Mohan	OS
Mohan	DBMS
Sunitha	DBMS Lab
Sunitha	DataStructur
Sunitha	OS
Sunitha	DBMS
Kavya	DBMS Lab
Kavya	DataStructur
Kavya	OS
Kavya	DBMS
Arjun	DBMS Lab
Arjun	DataStructur
Arjun	OS
Arjun	DBMS

```
20 rows in set (0.00 sec)
```

-- 7. JOIN with WHERE: Filtering joined results

```
-- Only CSE department courses, with faculty name
SELECT c.CourseDesc, f.FacName, f.Dept
FROM COURSE c
JOIN FACULTY f ON c.FacNo = f.FacNo
WHERE f.Dept = 'CSE';
```

```
mysql> -- Only CSE department courses, with faculty name
mysql> SELECT c.CourseDesc, f.FacName, f.Dept
-> FROM COURSE c
-> JOIN FACULTY f ON c.FacNo = f.FacNo
-> WHERE f.Dept = 'CSE';
```

CourseDesc	FacName	Dept
DBMS	Ramesh	CSE
DBMS Lab	Ramesh	CSE
OS	Mohan	CSE
DataStructur	Arjun	CSE

4 rows in set (0.00 sec)

-- 8. JOIN with GROUP BY: Aggregating across joined tables

```
-- Number of courses taught by each faculty member
SELECT f.FacName, COUNT(c.CourseNo) AS num_courses
FROM FACULTY f
LEFT JOIN COURSE c ON f.FacNo = c.FacNo
GROUP BY f.FacName
ORDER BY num_courses DESC;
```

```
mysql> -- Number of courses taught by each faculty member
mysql> SELECT f.FacName, COUNT(c.CourseNo) AS num_courses
-> FROM FACULTY f
-> LEFT JOIN COURSE c ON f.FacNo = c.FacNo
-> GROUP BY f.FacName
-> ORDER BY num_courses DESC;
```

FacName	num_courses
Ramesh	2
Mohan	1
Arjun	1
Sunitha	0
Kavya	0

5 rows in set (0.00 sec)

-- 9. JOIN with USING clause (shorthand when column names match)

```
SELECT s.Name, e.EnrollDate, c.CourseDesc
FROM STUDENT s
JOIN ENROLLMENT e USING (RegNo)
JOIN COURSE c USING (CourseNo);
```

```
mysql> SELECT s.Name, e.EnrollDate, c.CourseDesc
-> FROM STUDENT s
-> JOIN ENROLLMENT e USING (RegNo)
-> JOIN COURSE c USING (CourseNo);
```

Name	EnrollDate	CourseDesc
Aravind Kumar	2026-06-01	DBMS
Divya	2026-06-02	DBMS
Aravind Kumar	2026-06-01	OS
Karthik	2026-06-02	DataStructur
Meena	2026-06-03	DBMS Lab

5 rows in set (0.00 sec)

-- 10. Three-table JOIN with aliases

```
-- Full picture: student, the course, and who teaches it
SELECT s.Name AS Student, c.CourseDesc AS Course, f.FacName AS Faculty
FROM STUDENT s
JOIN ENROLLMENT e ON s.RegNo = e.RegNo
JOIN COURSE c ON e.CourseNo = c.CourseNo
JOIN FACULTY f ON c.FacNo = f.FacNo
ORDER BY s.Name;
```

```
mysql> -- Full picture: student, the course, and who teaches it
mysql> SELECT s.Name AS Student, c.CourseDesc AS Course, f.FacName AS Faculty
-> FROM STUDENT s
-> JOIN ENROLLMENT e ON s.RegNo = e.RegNo
-> JOIN COURSE c ON e.CourseNo = c.CourseNo
-> JOIN FACULTY f ON c.FacNo = f.FacNo
-> ORDER BY s.Name;
```

Student	Course	Faculty
Aravind Kumar	DBMS	Ramesh
Aravind Kumar	OS	Mohan
Divya	DBMS	Ramesh
Karthik	DataStructur	Arjun
Meena	DBMS Lab	Ramesh

5 rows in set (0.02 sec)